FPC CPP

Controlador Primera Persona / C# / Unity Asset

Documentación Técnica

v1.0 Unity 2022.3+

INTRODUCCIÓN

0. Introducción

Este documento cubre la lógica interna de los tres scripts que componen el sistema FPC CPP. Lo que encontrarás aquí es la descripción técnica de cada función, las operaciones físicas, algoritmos y matemáticas involucradas, para que puedas modificar o extender el "controlador" con pleno conocimiento de cómo funciona (No la Window en sí, haré otro Documento acerca de ello y actualizaré este mismo haciéndolo mucho más detallado y corrigiendo aquello que de hecho ya he cambiado en el código y ya no concuerda, a la par que incluyendo las nuevas cosas que implementaría, probablemente el otro Documento y la actualización de este vendrán con la primera actualización del Asset).

Arquitectura General

El asset está compuesto por tres scripts con responsabilidades claramente definidas:

Script	Rol	Notas
`PlayerConfiguration_ConfiguracionDelJugador.cs`	`Pure data ScriptableObject`	Sin lógica, sin herencia de MonoBehaviour. Almacena todos los parámetros configurables.
`PlayerControllerSetup_ControladorDelJugadorSetup.cs`	`Editor-only utility (#if UNITY_EDITOR)`	Registra los ejes del mando en el Input Manager cuando se carga el proyecto.
`PlayerController_ControladorDelJugador.cs`	`Main MonoBehaviour`	Toda la lógica de mecánicas en tiempo de ejecución se encuentra aqui: movimiento, salto, estados corporales, cámara, resistencia, zoom, objetos, deslizamiento, dash.

Relación Entre Scripts

PlayerConfiguration es el único archivo de datos. PlayerController lo lee en cada frame para tomar decisiones pero nunca escribe en él en tiempo de ejecución, excepto durante el Awake, para sincronizar alturas desde la escena. PlayerControllerSetup no interactúa con los otros dos en tiempo de ejecución.

> Importante: los ScriptableObjects no pueden referenciar objetos de la jerarquía de la escena. Por eso las referencias a la cámara, el soporte de cámara, todos los elementos del HUD y el punto de anclaje se asignan directamente en el Inspector del PlayerController.

1. PlayerConfiguration

ScriptableObject creado de la siguiente forma:

```
[CreateAssetMenu(fileName = "PlayerConfiguration_ConfiguracionDelJugador", menuName =
"PlayerConfiguration_ConfiguracionDelJugador", order = 0)]

public class PlayerConfiguration_ConfiguracionDelJugador : ScriptableObject
```

No hereda de MonoBehaviour, ni ejecuta Awake, Update ni ningún otro bucle de Unity. Es exclusivamente un contenedor de datos serializado que Unity persiste en disco como archivo .asset.

Para crearlo: clic derecho en la ventana Project - Create - PlayerConfiguration_ConfiguracionDelJugador. Luego arrástralo al campo Configuration_Configuracion del componente PlayerController en el Inspector.

> Si Configuration_Configuracion es null cuando Start() se ejecuta, el controlador llama a enabled = false y emite un Debug.LogError. Esto detiene todos los bucles (Update, FixedUpdate) sin propagar NullReferenceExceptions.

1.1 - Movimiento

Campo / Variable	Tipo	Por defecto	Descripción
AllowThePlayerToWalk	bool	true	Permiso principal para ApplyMovement(). Si es false, la función retorna inmediatamente.
AllowThePlayerToRun	bool	true	Verificado por ThePlayerIsRunningNow(). Si es false, el multiplicador de sprint nunca se aplica.
AllowThePlayerToRunWhileCrouching	bool	false	Permite correr mientras _currentBodyState == Crouching.
AllowThePlayerToRunWhileProne	bool	false	Permite correr mientras _currentBodyState == Prone.
AllowThePlayerToMoveInTheAir	bool	true	Si es false, GetMovementDirection() retorna _movementDirectionAtTakeoff en lugar del input en tiempo real.
BaseSpeedOfThePlayer	float	5	Velocidad base en unidades/segundo. Punto de partida para CalculateMovementSpeed().
SpeedMultiplierWhileRunning	float	1.8	Se aplica como: BaseSpeed * Multiplier cuando ThePlayerIsRunningNow() devuelve true.
HoldToRun	bool	true	true = GetKey() en cada frame. false = el input alterna _runToggleModeIsActive.

1.2 - Movimiento Omnidireccional

Multiplica la velocidad base según la dirección de movimiento y el estado corporal actual. Cuando está activo, CalculateMovementSpeed() calcula dos productos escalares e interpola entre el multiplicador direccional y el lateral.

Field / Variable	Type	Default	Description
EnableOmniDirectionalMovement	bool	true	Si es false, CalculateMovementSpeed() devuelve BaseSpeedOfThePlayer sin multiplicadores direccionales.
Forward / Backward / Lateral * 4 states	float	Var.	Un multiplicador por dirección (adelante/atrás/lateral) * estado corporal (Standing/Crouching/Prone/InAir). 12 variables en total.

1.3 - Estados Corporales

Field / Variable	Type	Default	Description
AllowThePlayerToCrouch	bool	true	'Bandera' de permiso verificada en ReadBodyStateInput(). Si es false, bloquea la transición a BodyState.Crouching.
HoldToCrouch	bool	false	Controla el modo mantener vs el modo palanca en ProcessKeyboardBodyStateInput().
AllowThePlayerToGoProne	bool	true	Permiso equivalente para BodyState.Prone.
HoldToProne	bool	false	Mantener vs Palanca para la postura acostado.
EnableCooldownBetweenBodyStateTransitions	bool	true	Si es true, ChangeBodyState() evalúa Time.time - _timeOfLastBodyStateTransition < Cooldown.
BodyStateTransitionCooldownTime	float	0.2	Segundos mínimos entre transiciones voluntarias.
SpeedOfTheCapsuleColliderTransition	float	8	Factor t del Lerp en UpdateCapsuleColliderTransition(): Time.deltaTime * Speed.
CapsuleColliderHeightWhileStanding	float	1.8	Sobreescrito en Awake() con la altura real de _playerCapsuleCollider.height de la escena.
CapsuleColliderHeightWhileCrouching	float	1.0	Altura objetivo del Lerp cuando _currentBodyState == Crouching.
CapsuleColliderHeightWhileProne	float	0.4	Altura objetivo del Lerp cuando _currentBodyState == Prone.

1.4 - Salto

Field / Variable	Type	Default	Description
AllowThePlayerToJump	bool	true	Permiso principal para ReadJumpInput() y ApplyJump().
HowManyJumps	int	1	Define cuántas veces puede saltar el jugador antes de necesitar tocar el suelo.
AllowThePlayerToJumpWhileProne	bool	true	Si es false, ApplyJump() retorna cuando _currentBodyState == Prone.
ForceAppliedWhenJumping	float	6	Se aplica con AddForce(Vector3.up * Force, ForceMode.VelocityChange). Ignora la masa.
GravityMultiplierDuringTheJump	float	2	Gravedad extra durante el ascenso: Physics.gravity * (Multiplier - 1f).
AdditionalGravityMultiplierDuringTheFall	float	1.5	Gravedad adicional durante la caída: Physics.gravity * (JumpMult * FallMult - 1f).
EnableVariableJump	bool	true	Activa la 'fase' de altura variable en ApplyJump().
MaximumHeldTimeOfTheJumpInput	float	0.3	Límite de _jumpInputHoldTime. Una vez superado, la fuerza extra se detiene.
ExtraForcePerSecondOfTheVariableJump	float	15	Se aplica con AddForce(Vector3.up * Force, ForceMode.Acceleration) mientras se mantiene pulsado.
LayersThatAreConsideredGround	LayerMask	—	Se pasa a Physics.CheckSphere() en GroundDetection() y al Raycast en ProcessPhysicalSliding(). Nota: CanStandUp() no usa esta máscara; usa Physics.DefaultRaycastLayers con la capa propia del jugador excluida.
RadiusOfTheGroundDetectionOverlapSphere	float	0.25	Radio de Physics.CheckSphere() en GroundDetection().
DownwardOffsetOfTheGroundDetectionOverlapSphere	float	0.05	Desplazamiento del centro de la esfera: transform.position + Vector3.down * Offset.
EnableCoyoteTime	bool	true	Permite saltar poco después de caminar por el borde de una plataforma.
DurationOfTheCoyoteTime	float	0.15	Ventana en segundos durante la cual se acepta el salto tardío.
EnableJumpBuffering	bool	true	Encola un salto pulsado ligeramente antes de aterrizar.
DurationOfTheJumpBuffering	float	0.15	Segundos que el input de salto en buffer permanece válido tras aterrizar.

1.5 - Resistencia

Field / Variable	Type	Default	Description
EnableTheStaminaSystem	bool	true	Interruptor maestro de la resistencia. Cuando es false, no se consume ni regenera resistencia.
MaximumPlayerStamina	float	100	Reserva máxima de la resistencia.
EnableStaminaCostWhenRunning	bool	true	Si correr consume resistencia o no.
StaminaCostPerSecondWhenRunning	float	15	Resistencia consumida por segundo al correr.
EnableStaminaCostWhenJumping	bool	true	Si saltar consume resistencia.
StaminaCostWhenJumping	float	10	Coste de resistencia por salto.
EnableStaminaCostWhenCrouching	bool	false	Si agacharse consume resistencia.
StaminaCostWhenCrouching	float	5	Coste de resistencia al transicionar a Agachado.
EnableStaminaCostWhenGoingProne	bool	false	Si acostarse consume resistencia o no.
StaminaCostWhenGoingProne	float	5	Coste de resistencia al transicionar a acostado.
EnableStaminaCostOnEachPostureTransition	bool	false	Si cada cambio de postura consume resistencia o no.
StaminaCostPerPostureTransition	float	3	Coste por transición cuando lo anterior está activado.
EnableExtraStaminaCostWhenJumpingFromProne	bool	true	Aplica penalización extra de resistencia al saltar desde la postura acostado.
ExtraStaminaCostWhenJumpingFromTheProneState	float	8	Coste extra añadido sobre el coste normal del salto.
DelayInSecondsBeforeStaminaStartsRegenerating	float	1.5	Segundos a esperar tras el último gasto de resistencia antes de que comience la regeneración.
StaminaRegenerationSpeedWhileThePlayerIsIdle	float	25	Resistencia por segundo al estar en reposo.
StaminaRegenerationSpeedWhileThePlayerWalks	float	10	Resistencia por segundo al estar caminando.
ShowTheStaminaBarOnTheHUD	bool	true	Alterna la visibilidad de la barra de resistencia en UpdateStaminaHUD().

1.6 - Balanceo de Cabeza

El desplazamiento continuo del balanceo se calcula con dos funciones trigonométricas. El eje Y usa sin(t) a frecuencia completa. El eje X usa cos(t * 0.5) a mitad de frecuencia y mitad de amplitud. Esto produce una oscilación elíptica: por cada dos ciclos verticales, ocurre un ciclo horizontal.

Field / Variable	Type	Default	Description
EnableTheHeadBobbingSystem	bool	true	Si es false, UpdateHeadBobbing() retorna inmediatamente.
EnableHeadBobbingWhileThePlayerWalks	bool	true	Activa la rama de intensidad/frecuencia al caminar.
IntensityOfHeadBobbingWhileThePlayerWalks	float	0.05	Amplitud de sin/cos al caminar.
FrequencyOfHeadBobbingWhileThePlayerWalks	float	12	Velocidad de avance del ‘temporizador’ al caminar.
EnableHeadBobbingWhileThePlayerRuns	bool	true	Activa la rama de intensidad/frecuencia al correr.
IntensityOfHeadBobbingWhileThePlayerRuns	float	0.1	Amplitud de sin/cos al correr.
FrequencyOfHeadBobbingWhileThePlayerRuns	float	16	Velocidad de avance del ‘temporizador’ al correr.
EnableReactiveHeadBobbingWhenJumpingAndLanding	bool	true	Activa el balanceo de impulso en el salto y el aterrizaje.
IntensityOfReactiveHeadBobbingWhenJumping	float	0.08	Vector3.back * Intensity asignado al saltar.
IntensityOfReactiveHeadBobbingWhenLanding	float	0.12	Vector3.down * Intensity asignado al aterrizar.
EnableReactiveHeadBobbingWhenCrouching	bool	true	Activa el impulso al transicionar a Agachado.
IntensityOfReactiveHeadBobbingWhenCrouching	float	0.07	Vector3.down * Intensity asignado al agacharse.
EnableReactiveHeadBobbingWhenGoingProne	bool	true	Activa el impulso al transicionar a Acostado.
IntensityOfReactiveHeadBobbingWhenGoingProne	float	0.12	Vector3.down * Intensity asignado al acostarse.
RetornoSpeedOfReactiveHeadBobbingToTheNeutralPosition	float	10	Factor t de Vector3.Lerp(_reactiveBobbingImpulse, Vector3.zero, Time.deltaTime * Speed) cada frame.
EnableReactiveHeadBobbingWhenSliding	bool	true	Activa el impulso de cabeza al iniciar el deslizamiento.
IntensityOfReactiveHeadBobbingWhenSliding	float	0.15	Magnitud del impulso al iniciar el deslizamiento.
EnableReactiveHeadBobbingWhenDashing	bool	true	Activa el impulso de cabeza al hacer dash.
IntensityOfReactiveHeadBobbingWhenDashing	float	0.1	Calculado con InverseTransformDirection, relativo a la dirección del dash.
EnableBreathingEffect	bool	true	Activa el pitch de respiración + balanceo de traslación impulsado por la resistencia.
BreathsPerMinuteAtRest	float	14	Frecuencia de respiración con la resistencia completa (rango 8–40 BPM).
BreathsPerMinuteWhenExhausted	float	30	Frecuencia de respiración con la resistencia en cero (rango 8–40 BPM).
BreathingInhaleFraction	float	0.3	Fracción de un ciclo de respiración dedicada a la inhalación (rango 0.1–0.9).
BreathingBasePitchIntensity	float	3	Balanceo del pitch de la cámara en grados con la resistencia completa.
BreathingExhaustedPitchIntensity	float	5	Balanceo del pitch de la cámara en grados con la resistencia en cero.

Field / Variable	Type	Default	Description
BreathingBaseTranslationIntensity	float	0.05	Traslación Y de la cámara con la resistencia completa.
BreathingExhaustedTranslationIntensity	float	0.05	Traslación Y de la cámara con la resistencia en cero.
BreathingExhaustionFadeInSpeed	float	1.5	Tamaño de entrada por segundo en MoveTowards con el que sube la influencia del agotamiento.
BreathingExhaustionFadeOutSpeed	float	0.8	Tamaño de entrada por segundo en MoveTowards con el que baja la influencia del agotamiento.

1.7 - Cámara

Field / Variable	Type	Default	Description
BaseFieldOfViewOfTheCamera	float	75	FOV objetivo del Lerp cuando _zoomIsActive es false. Se inicializa en Awake().
EnableSpeedFOVEffect	bool	true	Añade un ligero aumento del FOV cuando el jugador supera un cierto umbral de velocidad.
SpeedFOVStartPercent	float	0.8	Fracción de BaseSpeed a partir de la cual comienza el efecto del FOV.
SpeedFOVMaxPercent	float	1.6	Fracción de velocidad a la que el efecto del FOV se aplica completamente.
SpeedFOVMaxMultiplier	float	1.15	Multiplicador máximo del FOV aplicado por el efecto de la velocidad.
SpeedFOVTransitionSpeed	float	6	Velocidad del Lerp para la transición del FOV por velocidad.
HorizontalMouseSensitivity	float	2	Multiplicador de Input.GetAxis("Mouse X") o rsH * sensH.
VerticalMouseSensitivity	float	2	Multiplicador de Input.GetAxis("Mouse Y") o rsV * sensV.
UpperVerticalLimitOfTheCamera	float	80	Límite superior de _cameraAccumulatedVerticalRotationAngle.
LowerVerticalLimitOfTheCamera	float	80	Límite inferior de _cameraAccumulatedVerticalRotationAngle.
ShowTheCrosshairOnTheHUD	bool	true	Se pasa a CrosshairOnHUD.SetActive() en UpdateCrosshairVisibility().
HeightOfTheCameraSupportWhileStanding	float	1.6	Sobreescrito en Awake() con la Y real de CameraSupport.localPosition.
HeightOfTheCameraSupportWhileCrouching	float	0.7	Objetivo Y del Lerp cuando _currentBodyState == Crouching.
HeightOfTheCameraSupportWhileProne	float	0.2	Objetivo Y del Lerp cuando _currentBodyState == Prone.
SpeedOfTheCameraHeightTransition	float	8	Factor t de Mathf.Lerp para la altura: Time.deltaTime * Speed.

1.8 - Zoom

Field / Variable	Type	Default	Description
EnableTheZoomSystem	bool	true	Permiso central para ReadZoomInput() y UpdateZoom().
HoldToZoom	bool	true	true = _zoomIsActive = ZoomInputHeld(). false = palanca con ZoomInputPressed().
AllowZoomWhileHoldingAnObject	bool	true	Si es false y _currentlyHeldObject != null, ReadZoomInput() retorna.
CameraFieldOfViewDuringZoom	float	40	FOV objetivo del Lerp cuando _zoomIsActive es true.
SpeedOfTheFieldOfViewTransitionDuringZoom	float	8	Factor t de Mathf.Lerp del FOV en UpdateZoom().
ReduceSensitivityDuringZoom	bool	true	Si es true, ReadCameraInput() multiplica sensH y sensV por el multiplicador.
SensitivityMultiplierDuringZoom	float	0.5	sensH *= Multiplier cuando el zoom está activo.

1.9 - Interacción con Objetos

Field / Variable	Type	Default	Description
EnableTheObjectInteractionSystem	bool	true	Permiso principal para ReadObjectInteractionInput().
TagOfPickableObjects	string	Recogible	Se pasa a hit.collider.CompareTag() en TryToPickUpTheObject().
MaximumDistanceToPickUpAnObject	float	3	Longitud del Raycast desde MainCamera_CamaraPrincipal.transform.position.
DisableTheObjectColliderWhenPickingItUp	bool	true	Si es true, PickUpObject() desactiva el Collider del objeto.
SpeedOfTheObjectMovementTowardsTheAnchorPoint	float	15	Factor t de Vector3.Lerp en UpdateHeldObjectPosition().
HoldToRotateTheObject	bool	false	Alterna entre modo mantener y palanca en ReadObjectInteractionInput().
RotationSpeedOfTheObjectInHand	float	90	Grados por segundo en RotateObjectInHand().
TakeIntoAccountTheMassOfTheObjectWhenThrowingIt	bool	false	Alterna entre ForceMode.VelocityChange y ForceMode.Impulse.
MinimumObjectThrowForce	float	5	Fuerza aplicada cuando el lanzamiento se ejecuta sin haberlo cargado.
MaximumObjectThrowForce	float	20	Fuerza aplicada cuando el lanzamiento ejecuta estando completamente cargado.
MaximumChargeTimeOfTheObjectThrow	float	1.5	Segundos necesarios para alcanzar la fuerza máxima del lanzamiento.

1.10 - Movimiento Avanzado: Deslizamiento y Dash

Deslizamiento

Field / Variable	Type	Default	Description
EnableTheSlidingSystem	bool	true	Interruptor maestro para la mecánica del deslizamiento.
UseRealPhysicsInSliding	bool	false	true = deslizamiento físico. false = deslizamiento arcade.
AllowJumpingDuringSliding	bool	true	Permite saltar mientras _playerIsSliding.
AllowInterruptingSliding	bool	true	Permite al jugador cancelar un deslizamiento anticipadamente.
RecoveryTimeAfterSliding	float	0.4	Segundos que dura el estado de recuperación forzado tras terminar el deslizamiento.
AllowRunningImmediatelyAfterSliding	bool	false	Omite el período de recuperación.
EnableStaminaCostWhenSliding	bool	true	Si deslizarse consume resistencia o no.
StaminaCostWhenSliding	float	15	Resistencia consumida al iniciar el deslizamiento.
AllowSlidingOnAnySurface	bool	true	Cuando es false, restringe el deslizamiento a pendientes más pronunciadas.
DurationOfSlidingInArcadeMode	float	0.8	Duración fija del deslizamiento en modo arcade (segundos).
InitialSpeedMultiplierOfArcadeSliding	float	2.5	Multiplicador de velocidad aplicado al inicio del deslizamiento arcade.
MinimumSpeedToKeepSliding	float	1	Velocidad mínima por debajo de la cual el deslizamiento arcade termina antes.
FrictionDuringPhysicalSliding	float	0.3	Fricción aplicada en el modo de deslizamiento físico. AddForce(-horizontalVel * Friction).
SlopeAccelerationMultiplierDuringSliding	float	1.5	Magnitud del AddForce(slopeDirection * Multiplier) en pendientes.
ReduceCameraSensitivityDuringSliding	bool	true	Si es true, ReadCameraInput() aplica el multiplicador durante el deslizamiento.
CameraSensitivityMultiplierDuringSliding	float	0.5	sensH *= Multiplier durante el deslizamiento.

Dash

Field / Variable	Type	Default	Description
EnableTheDashSystem	bool	true	Permiso principal para ReadDashInput() y TryExecuteDash().
AllowDashInTheAir	bool	true	Si es false, TryExecuteDash() retorna cuando !_playerIsOnGround.
DashForce	float	15	velocity.x = dashDir.x * Force; velocity.z = dashDir.z * Force. Asignación directa, no AddForce.
DurationOfTheDashImpulse	float	0.2	Segundos del yield return new WaitForSeconds() en ExecuteDash(). Bloquea ApplyMovement().
CooldownBetweenDashUses	float	1	TryExecuteDash() retorna si _timeSinceLastDash < Cooldown.
RequireDoublePressForDashOnGamepad	bool	false	Activa la lógica de doble pulsación en ReadDashInput() para el mando.
MaximumTimeBetweenTheTwoPressesForDashDoublePress	float	0.3	Ventana máxima entre la primera y la segunda pulsación. Si se supera, _waitingForSecondDashButtonPress = false.

Field / Variable	Type	Default	Description
AllowDashDuringSliding	bool	true	Si es false, TryExecuteDash() retorna cuando se está deslizando o en recuperación.
EnableStaminaCostWhenUsingDash	bool	true	Si es true, TryExecuteDash() verifica y dreduce resistencia.
StaminaCostWhenUsingDash	float	20	Si CurrentStamina < Cost, TryExecuteDash() retorna sin ejecutarse.

1.11 - Input: Teclado

Field / Variable	Type	Default	Description
KeyboardKeyToMoveForward	KeyCode	W	inputVertical += 1f en GetMovementDirection().
KeyboardKeyToMoveBackward	KeyCode	S	inputVertical -= 1f.
KeyboardKeyToMoveLeft	KeyCode	A	inputHorizontal -= 1f.
KeyboardKeyToMoveRight	KeyCode	D	inputHorizontal += 1f.
KeyboardKeyToRun	KeyCode	LeftShift	GetKey() en RunInputHeld() y GetKeyDown() en ReadRunInput().
KeyboardKeyToJump	KeyCode	Space	GetKeyDown() / GetKey() / GetKeyUp() en los tres métodos del input de salto.
KeyboardKeyToCrouch	KeyCode	C	GetKeyDown() para iniciar el agachado o deslizamiento en ProcessKeyboardBodyStateInput().
KeyboardKeyToGoProne	KeyCode	V	GetKeyDown() o GetKey() según el modo mantener/palanca.
KeyboardKeyForZoom	KeyCode	Z	También acepta Input.GetMouseButton(1) como alternativa.
KeyboardKeyToPickUpOrDropAnObject	KeyCode	E	GetKeyDown() en PickUpOrDropInputPressed().
KeyboardKeyToActivateObjectRotationInHand	KeyCode	R	GetKeyDown() o GetKey() según HoldToRotateTheObject.
Keyboard keys to rotate object (×4)	KeyCode	Arrows	verticalRotationInput y horizontalRotationInput en RotateObjectInHand().
MouseButtonToThrowTheObjectInHand	int	0	GetMouseButton(0) en ThrowInputHeld(), GetMouseButtonUp(0) en ThrowInputReleased().
KeyboardKeyForDash	KeyCode	Q	GetKeyDown() en DashInputPressed().

1.12 - Input: Mando

Field / Variable	Type	Default	Description
LeftStickDeadZone	float	0.2	if (Mathf.Abs(inputHorizontal) < DeadZone) inputHorizontal = 0f en GetMovementDirection().
RightStickDeadZone	float	0.2	El mismo filtro para rsH y rsV en ReadCameraInput().
GamepadButtonToJump	KeyCode	JoystickButton0	Xbox: A / PS: Cruz.
GamepadButtonToCrouchAndGoProne	KeyCode	JoystickButton1	Xbox: B / PS: Círculo. Pulsación corta = agacharse, pulsación larga = acostarse.
GamepadButtonToPickUpOrDropAnObject	KeyCode	JoystickButton2	Xbox: X / PS: Cuadrado.
GamepadButtonToActivateObjectRotationInHand	KeyCode	JoystickButton5	Xbox: RB / PS: R1.
GamepadButtonForDash	KeyCode	JoystickButton4	Xbox: LB / PS: L1.
GamepadButtonToRun	KeyCode	JoystickButton8	Xbox: L3 / PS: L3.
GamepadButtonForZoom	KeyCode	JoystickButton9	También acepta _LTZoomPressedThisFrame.
GamepadButtonToThrowTheObjectInHand	KeyCode	JoystickButton10	También acepta _RTIsActive y _RTReleasedThisFrame.
ButtonHoldTimeToBeConsideredAsProne	float	0.4	Umbral del _buttonHoldTimeForCrouchOrProneOnGamepad para interpretar la pulsación como acostarse.

2. PlayerControllerSetup

Script envuelto en un bloque #if UNITY_EDITOR. No existe en tiempo de ejecución ni en builds. Se compila exclusivamente para el Editor de Unity. El archivo debe residir dentro de una carpeta llamada Editor en cualquier lugar del proyecto.

2.1 - Constructor Estático [InitializeOnLoad]

La clase tiene el atributo [InitializeOnLoad], que hace que Unity ejecute el constructor estático automáticamente cuando el Editor se carga o recompila:

```
[InitializeOnLoad]
public static class PlayerControllerSetup_ControladorDelJugadorSetup
{
    // ═══════════════════════════════════════════════════════════════════════════
    #region INICIALIZACIÓN / INITIALIZATION
    // ═══════════════════════════════════════════════════════════════════════════

    static PlayerControllerSetup_ControladorDelJugadorSetup()
    {
        RegisterGamepadAxesIfMissing();
    }

    #endregion
    // ═══════════════════════════════════════════════════════════════════════════
```

El constructor llama a RegisterGamepadAxesIfMissing(), que también está expuesto como elemento del menú:

```
    [MenuItem("FPC - CPP / Register Gamepad Axes - Registrar Ejes del Mando")]

    public static void RegisterGamepadAxesIfMissing()
```

Si los ejes desaparecen del Input Manager, pueden volver a registrarse manualmente desde ese menú sin necesidad de reimportar el asset.

2.2 - RegisterAxis()

Antes de registrar cualquier eje, la función itera el array m_Axes para verificar que no exista ya un eje con el mismo nombre. Si existe, retorna false sin modificar nada:

```
        for (int i = 0; i < axesProperty.arraySize; i++)
        {
            if (axesProperty.GetArrayElementAtIndex(i).FindPropertyRelative("m_Name").stringValue == name)
                return false;
        }
```

Si no existe, incrementa arraySize, accede al nuevo elemento por índice y asigna todas las propiedades serializadas. La zona muerta es 0.19f, sensibilidad 1f para todos los ejes. Nota: esta es la zona muerta almacenada en el asset del Input Manager y aplicada por el sistema de input de Unity. Es distinta de LeftStickDeadZone y RightStickDeadZone que son umbrales separados aplicados en código dentro de GetMovementDirection() y ReadCameraInput().

> El script nunca modifica ejes existentes. Solo añade los que faltan. No hay riesgo de sobreescribir configuraciones previas del proyecto.

Parámetros de RegisterAxis():

Parameter	Type	Description
axesProperty	SerializedProperty	Referencia al array m_Axes en el InputManager.asset.
name	string	Nombre del eje tal como aparecerá en el código (ej. "RSHorizontal").
joyAxisNum	int	Número del eje del joystick de Unity (base 1; almacenado internamente como num − 1).
inverted	bool	Si es true el eje se invierte. Se usa para los ejes verticales del stick (RSVertical, DpadVertical).

2.3 - Ejes Registrados

Nombre del Eje	Eje de Joystick	Description
RSHorizontal	Axis 4	Joystick derecho horizontal. Cámara H. Leído en ReadCameraInput() con Input.GetAxisRaw().
RSVertical	Axis 5	Joystick derecho vertical. Cámara V. Registrado con invert = true.
DpadHorizontal	Axis 6	Cruceta H. Leído en RotateObjectInHand() con Input.GetAxisRaw().
DpadVertical	Axis 7	Cruceta V. Leído en RotateObjectInHand() con Input.GetAxisRaw(). Registrado con invert = true.
LT	Axis 9	Gatillo analógico izquierdo. Leído en DetectGamepadTriggers(). Umbral de activación: > 0.3f.
RT	Axis 10	Gatillo analógico derecho. Leído en DetectGamepadTriggers(). Umbral: > 0.3f.

3. PlayerController

MonoBehaviour principal decorado con [RequireComponent(typeof(Rigidbody))] y [RequireComponent(typeof(CapsuleCollider))]. Las referencias de escena que no pueden vivir en el ScriptableObject se asignan en el Inspector: Configuration_Configuracion, CameraSupport_SoporteDeLaCamara, MainCamera_CamaraPrincipal, las imágenes de la barra de resistencia, el crosshair y el punto de anclaje del objeto.

3.1 - Awake()

Se ejecuta antes que cualquier Start(), incluso si el GameObject está desactivado. Obtención de componentes y configuración del Rigidbody:

```
_playerRigidbody = GetComponent<Rigidbody>();
_playerCapsuleCollider = GetComponent<CapsuleCollider>();
_originalCapsuleRadius = _playerCapsuleCollider.radius;

_playerRigidbody.freezeRotation = true;

_playerRigidbody.interpolation = RigidbodyInterpolation.None;

PlayerCurrentStamina = Configuration_Configuracion.MaximumPlayerStamina;
```

freezeRotation = true congela la rotación del Rigidbody en los tres ejes. Sin esto, cualquier ContactPoint asimétrico en una colisión aplicaría un torque y el personaje se inclinaría, caería o giraría. Con esto activo, toda la rotación la controla exclusivamente el script mediante transform.Rotate() y Quaternion.Euler().

Sincronización de alturas desde la escena al ScriptableObject:

```
if (CameraSupport_SoporteDeLaCamara != null)
    _originalYPositionOfCameraSupport = CameraSupport_SoporteDeLaCamara.localPosition.y;

if (Configuration_Configuracion != null)
{
    Configuration_Configuracion.HeightOfTheCameraSupportWhileStanding = _originalYPositionOfCameraSupport;
    Configuration_Configuracion.CapsuleColliderHeightWhileStanding = _playerCapsuleCollider.height;
}
```

La escena tiene prioridad sobre el ScriptableObject: si el soporte de cámara o el CapsuleCollider se modificaron en el Inspector, Awake captura esos valores en tiempo de ejecución y los escribe en el asset, evitando discrepancias entre el asset de datos y la escena real. Awake también inicializa PlayerCurrentStamina a MaximumPlayerStamina, establece _currentCameraFieldOfView a BaseFieldOfViewOfTheCamera y lo aplica inmediatamente a MainCamera_CamaraPrincipal.fieldOfView, y bloquea y oculta el cursor (Cursor.lockState = CursorLockMode.Locked, Cursor.visible = false).

Para evitar que el primer frame dispare falsamente el evento de aterrizaje, ambas variables de suelo se inicializan en true:

```
        _playerWasOnGroundLastFrame = true;
        _playerIsOnGround = true;
        _remainingJumps = Configuration_Configuracion.HowManyJumps;
```

Nota: el cooldown del dash se inicializa en Start(), no en Awake(), con _timeSinceLastDash = CooldownBetweenDashUses + 1f, para que el dash esté disponible desde el primer frame.

3.2 - Start()

Se ejecuta una vez en el primer frame, después de que todas las llamadas a Awake han completado. Valida que Configuration_Configuracion esté asignado, si es null, llama a enabled = false y registra un error, deteniendo todos los bucles sin propagar NullReferenceExceptions. Si tiene éxito, llama a UpdateCrosshairVisibility() y UpdateStaminaHUD() para configurar el estado inicial del HUD, guarda el ancho original en píxeles de la barra de resistencia como referencia de relleno, e inicializa _timeSinceLastDash = CooldownBetweenDashUses + 1f para que el dash esté disponible desde el primer frame.

3.3 - Update()

Se ejecuta en cada frame. No contiene lógica de físicas y no modifica el Rigidbody. Orquesta las llamadas en un orden fijo:

```
        if (Configuration_Configuracion == null) return;

        DetectActiveInputDevice();
        DetectGamepadTriggers();

        ReadCameraInput();
        ReadBodyStateInput();
        ReadJumpInput();
        ReadRunInput();
        ReadZoomInput();
        ReadObjectInteractionInput();
        ReadDashInput();

        if (_playerIsSliding && Configuration_Configuracion.AllowInterruptingSliding && _timeSinceSlidingStarted
> 0f)
            _interruptSlideInputCached = _interruptSlideInputCached || InputToInterruptSliding();

        UpdateCoyoteTimeTimer();
        UpdateJumpBufferTimer();
        UpdateDashCooldownTimer();

        UpdatePostSlideRecovery();
        UpdateCapsuleColliderTransition();
        UpdateCameraHeight();
        UpdateHeadBobbing();
        UpdateZoom();
        UpdateStaminaHUD();

        if (MainCamera_CamaraPrincipal != null)
            MainCamera_CamaraPrincipal.fieldOfView = _currentCameraFieldOfView;

        EnforceCanStandUp();
        _cachedMovementDirection = GetMovementDirection();
```

GetMovementDirection() se llama al final de Update y se almacena en el caché en _cachedMovementDirection. FixedUpdate consume ese valor cacheado en ApplyMovement(). Esto garantiza que tanto la rotación como las físicas usen el mismo snapshot de dirección por frame, evitando inconsistencias cuando Update y FixedUpdate se ejecutan a diferentes tasas.

3.4 - FixedUpdate()

Bucle de físicas a tasa fija (Time.fixedDeltaTime = 0.02s por defecto). Todo lo que asigna velocidad o llama a AddForce en el Rigidbody va aquí. El orden interno es deliberado:

```
GroundDetection();
ApplyGravity();
ApplyMovement();
ApplyJump();
ExecuteSliding();
UpdateStamina();
```

3.5 - GroundDetection()

Usa Physics.CheckSphere en lugar de Raycast para evitar falsos negativos en bordes irregulares o bordes simples de plataformas:

```
_playerWasOnGroundLastFrame = _playerIsOnGround;

Vector3 detectionPoint = transform.position + Vector3.down *
Configuration_Configuracion.DownwardOffsetOfTheGroundDetectionOverlapSphere;
_playerIsOnGround = Physics.CheckSphere(
    detectionPoint,
    Configuration_Configuracion.RadiusOfTheGroundDetectionOverlapSphere,
    Configuration_Configuracion.LayersThatAreConsideredGround,
    QueryTriggerInteraction.Ignore);
```

QueryTriggerInteraction.Ignore excluye los Trigger Colliders de la detección. Sin este parámetro, cualquier zona trigger en el suelo activaría _playerIsOnGround incluso si el jugador estuviera en el aire por encima.

La comparación entre el estado del frame actual y el anterior detecta dos flancos:

- Flanco ascendente (false - true) - llama a OnLanding().
- Flanco descendente sin salto (true - false && !_playerIsPerformingJump) - asigna _remainingCoyoteTime = DurationOfTheCoyoteTime.

OnLanding() reinicia _remainingJumps, limpia las variables del estado de salto, cancela cualquier deslizamiento activo, asigna el impulso del balanceo reactivo de aterrizaje y llama a RestoreBodyStateOnLanding(). Esto última reconstruye la postura correcta, agachado/acostado/de pie, según qué estado de palanca/mantener estaba activo antes del salto.

3.6 - ApplyGravity()

Unity aplica Physics.gravity a todos los Rigidbodies con useGravity = true. Este método añade fuerza extra encima usando ForceMode.Acceleration, que aplica la aceleración independientemente de la masa:

```
        if (_playerIsOnGround) return;

        if (_playerRigidbody.velocity.y < 0f)
            _playerRigidbody.AddForce(
                Physics.gravity * (Configuration_Configuracion.GravityMultiplierDuringTheJump *
Configuration_Configuracion.AdditionalGravityMultiplierDuringTheFall - 1f),
                ForceMode.Acceleration);
        else
            _playerRigidbody.AddForce(
                Physics.gravity * (Configuration_Configuracion.GravityMultiplierDuringTheJump - 1f),
                ForceMode.Acceleration);
```

El -1 en ambas fórmulas es necesario: Unity ya aplica Physics.gravity una vez. Si se multiplica directamente por el multiplicador sin restar 1, la gravedad se aplicaría dos veces en la fase normal. La resta compensa esto.

Resultado: durante el ascenso (velocity.y >= 0), la fuerza extra añadida es (GravityMultiplierDuringTheJump - 1) * gravity, por lo que la gravedad efectiva total es GravityMultiplierDuringTheJump * gravity. Durante el descenso (velocity.y < 0), la fuerza extra es (GravityMultiplierDuringTheJump * AdditionalGravityMultiplierDuringTheFall - 1) * gravity, haciendo que la gravedad efectiva total sea GravityMultiplierDuringTheJump * AdditionalGravityMultiplierDuringTheFall * gravity.

> Se elige ForceMode.Acceleration sobre ForceMode.Force para que el efecto sea independiente de la masa del Rigidbody. Con Force, cambiar la masa del personaje alteraría la curva del salto.

3.7 - ApplyMovement(), GetMovementDirection(), CalculateMovementSpeed()

ApplyMovement()

Asigna la velocidad directamente al Rigidbody mientras preserva el componente Y:

```
        if (!Configuration_Configuracion.AllowThePlayerToWalk) return;
        if (_playerIsSliding) return;
        if (_playerIsPerformingDash) return;

        Vector3 movementDirection = _cachedMovementDirection;
        float finalSpeed = CalculateMovementSpeed(movementDirection);

        float velocityY = (_playerIsOnGround && !_playerIsPerformingJump)
            ? Mathf.Min(_playerRigidbody.velocity.y, 0f)
            : _playerRigidbody.velocity.y;

        _playerRigidbody.velocity = new Vector3(
            movementDirection.x * finalSpeed,
            velocityY,
            movementDirection.z * finalSpeed);

        if (_playerIsOnGround)
            _movementDirectionAtTakeoff = movementDirection;
```

Asignar directamente .x y .z (en lugar de AddForce) hace que el movimiento sea inmediato: cada frame la velocidad horizontal refleja exactamente el input, sin inercia acumulada. Soltar el input detiene el movimiento inmediatamente, el comportamiento esperado para un FPS por ejemplo.

> AddForce acumularía velocidad frame a frame. Con asignación directa, soltar el input detiene el movimiento inmediatamente.

GetMovementDirection()

Construye el vector de dirección en el espacio local del jugador:

```
if (!_playerIsOnGround && !Configuration_Configuracion.AllowThePlayerToMoveInTheAir)
    return _movementDirectionAtTakeoff;

float inputHorizontal = 0f;
float inputVertical = 0f;

if (_isUsingGamepad)
{
    inputHorizontal = Input.GetAxisRaw("Horizontal");
    inputVertical = Input.GetAxisRaw("Vertical");
    if (Mathf.Abs(inputHorizontal) < Configuration_Configuracion.LeftStickDeadZone) inputHorizontal = 0f;
    if (Mathf.Abs(inputVertical) < Configuration_Configuracion.LeftStickDeadZone) inputVertical = 0f;
}
else
{
    if (Input.GetKey(Configuration_Configuracion.KeyboardKeyToMoveForward)) inputVertical += 1f;
```

```
    if (Input.GetKey(Configuration_Configuracion.KeyboardKeyToMoveBackward)) inputVertical -= 1f;
    if (Input.GetKey(Configuration_Configuracion.KeyboardKeyToMoveRight)) inputHorizontal += 1f;
    if (Input.GetKey(Configuration_Configuracion.KeyboardKeyToMoveLeft)) inputHorizontal -= 1f;
}

if (inputHorizontal == 0f && inputVertical == 0f) return Vector3.zero;

Vector3 direction = transform.right * inputHorizontal + transform.forward * inputVertical;
direction.y = 0f;
return direction.normalized;
```

`direction.y = 0f` es crítico. Sin él, si la cámara apunta hacia arriba o abajo, el componente forward tendría una Y distinta de cero y el personaje se movería en diagonal respecto al suelo.

`GetAxisRaw()` devuelve el valor del eje sin el suavizado interno de Unity que aplica `GetAxis()`, eliminando el lag perceptible al iniciar y detener el movimiento.

CalculateMovementSpeed()

Calcula la velocidad final aplicando los multiplicadores de omnidireccionalidad:

```
        float baseSpeed = Configuration_Configuracion.BaseSpeedOfThePlayer * (playerIsRunning ?
Configuration_Configuracion.SpeedMultiplierWhileRunning : 1f);

        if (!Configuration_Configuracion.EnableOmniDirectionalMovement) return baseSpeed;

        float dotForward = Vector3.Dot(transform.forward, movementDirection);
        float dotLateral = Mathf.Abs(Vector3.Dot(transform.right, movementDirection));

        float directionalMultiplier;
        float lateralMultiplier;

        switch (_currentBodyState)
        {
            case BodyState.Crouching:
                directionalMultiplier = dotForward >= 0f ?
Configuration_Configuracion.ForwardOmnidirectionalMultiplierWhileCrouching :
Configuration_Configuracion.BackwardOmnidirectionalMultiplierWhileCrouching;
                lateralMultiplier = Configuration_Configuracion.LateralOmnidirectionalMultiplierWhileCrouching;
                break;
            case BodyState.Prone:
                directionalMultiplier = dotForward >= 0f ?
Configuration_Configuracion.ForwardOmnidirectionalMultiplierWhileProne :
Configuration_Configuracion.BackwardOmnidirectionalMultiplierWhileProne;
                lateralMultiplier = Configuration_Configuracion.LateralOmnidirectionalMultiplierWhileProne;
                break;
            case BodyState.InTheAir:
                directionalMultiplier = dotForward >= 0f ?
Configuration_Configuracion.ForwardOmnidirectionalMultiplierWhileInTheAir :
Configuration_Configuracion.BackwardOmnidirectionalMultiplierWhileInTheAir;
                lateralMultiplier = Configuration_Configuracion.LateralOmnidirectionalMultiplierWhileInTheAir;
                break;
            default:
                directionalMultiplier = dotForward >= 0f ?
Configuration_Configuracion.ForwardOmnidirectionalMultiplierWhileStanding :
Configuration_Configuracion.BackwardOmnidirectionalMultiplierWhileStanding;
                lateralMultiplier = Configuration_Configuracion.LateralOmnidirectionalMultiplierWhileStanding;
                break;
        }

        return baseSpeed * Mathf.Lerp(directionalMultiplier, lateralMultiplier, dotLateral);
```

El Lerp usa `dotLateral` como `t`. Cuando el movimiento es recto (`dotLateral ≈ 0`), devuelve el multiplicador direccional. Cuando es completamente lateral (`dotLateral ≈ 1`), devuelve el multiplicador lateral. El movimiento diagonal produce un valor intermedio proporcional.

3.8 - Estados Corporales

El enum privado BodyState tiene 4 valores: Standing, Crouching, Prone, InTheAir. El estado activo vive en _currentBodyState.

Diferencias entre Teclado y Mando

En teclado, agacharse y acostarse tienen KeyCodes separados. En mando, el mismo botón (JoystickButton1) gestiona ambas posturas. La distinción se hace acumulando _buttonHoldTimeForCrouchOrProneOnGamepad con Time.deltaTime en cada frame que el botón está pulsado:

```
        if (buttonHeld)
            _buttonHoldTimeForCrouchOrProneOnGamepad += Time.deltaTime;


        if (buttonHeld && _buttonHoldTimeForCrouchOrProneOnGamepad >=
Configuration_Configuracion.ButtonHoldTimeToBeConsideredAsProne && !_gamepadButtonAlreadyInterpretedAsProne)
        {
            _gamepadButtonAlreadyInterpretedAsProne = true;


        if (buttonReleased)
        {
            if (!_gamepadButtonAlreadyInterpretedAsProne)
            {

                if (ThePlayerIsRunningNow() && Configuration_Configuracion.EnableTheSlidingSystem &&
_playerIsOnGround)
                {
                    StartSliding();
                }
                else - Process as Crouch


            _buttonHoldTimeForCrouchOrProneOnGamepad = 0f;
            _gamepadButtonAlreadyInterpretedAsProne = false;
```

Si ambas posturas se activan en el mismo frame (posible en teclado con teclas separadas), gana la más reciente usando los timestamps _timestampOfLastCrouchInputPulse y _timestampOfLastProneInputPulse.

CanStandUp()

```
        float difference = Configuration_Configuracion.CapsuleColliderHeightWhileStanding -
_playerCapsuleCollider.height;
        if (difference <= 0.05f) return true;

        int maskWithoutPlayer = Physics.DefaultRaycastLayers & ~(1 << gameObject.layer);

        bool result = !Physics.SphereCast(
            transform.position + Vector3.up * (_playerCapsuleCollider.center.y),
            _playerCapsuleCollider.radius,
            Vector3.up,
            out RaycastHit hit,
            Configuration_Configuracion.CapsuleColliderHeightWhileStanding - _playerCapsuleCollider.radius * 2f,
            maskWithoutPlayer,
            QueryTriggerInteraction.Ignore);
```

El SphereCast parte del centro de la cápsula (center.y) con el radio propio del collider, barriendo hacia arriba una distancia de CapsuleColliderHeightWhileStanding - radio * 2f. La máscara es Physics.DefaultRaycastLayers con la capa propia del jugador excluida, para que el collider no colisione consigo mismo. Si se detecta geometría, el jugador no puede levantarse. El umbral de retorno anticipado 0.05f previene falsos negativos cuando el collider ya está a la altura de pie o cerca de ella.

ChangeBodyState(BodyState newState, bool voluntaryChange)

El parámetro voluntaryChange determina si se verifican el cooldown y los costes de resistencia. Si es false (aterrizaje, deslizamiento forzado, cambio de estado provocado por el salto), ambas verificaciones se omiten. Si es true (input del jugador), se aplica el cooldown y se reduce la resistencia si corresponde claro.

3.9 - UpdateCapsuleColliderTransition()

```
        float targetHeight = _currentBodyState switch
        {
            BodyState.Crouching => Configuration_Configuracion.CapsuleColliderHeightWhileCrouching,
            BodyState.Prone => Configuration_Configuracion.CapsuleColliderHeightWhileProne,
            _ => Configuration_Configuracion.CapsuleColliderHeightWhileStanding
        };

        float targetRadius = _currentBodyState == BodyState.Prone
            ? targetHeight * 0.5f
            : _originalCapsuleRadius;

        if (Mathf.Abs(_playerCapsuleCollider.height - targetHeight) < 0.001f)
        {
            _playerCapsuleCollider.height = targetHeight;
            _playerCapsuleCollider.radius = targetRadius;
            _playerCapsuleCollider.center = new Vector3(0f, targetHeight * 0.5f, 0f);
            return;
        }

        _playerCapsuleCollider.radius = Mathf.Lerp(_playerCapsuleCollider.radius, targetRadius, Time.deltaTime *
Configuration_Configuracion.SpeedOfTheCapsuleColliderTransition);
        _playerCapsuleCollider.height = Mathf.Lerp(_playerCapsuleCollider.height, targetHeight, Time.deltaTime *
Configuration_Configuracion.SpeedOfTheCapsuleColliderTransition);
        _playerCapsuleCollider.center = new Vector3(0f, _playerCapsuleCollider.height * 0.5f, 0f);
```

El centro siempre se recalcula como height * 0.5f. El CapsuleCollider está anclado en su centro geométrico: si el centro no se mueve cuando cambia la altura, el collider crece hacia arriba y hacia abajo simultáneamente, provocando que el jugador flote o se hunda. En el estado Acostado, el radio del collider también se interpola hacia targetHeight * 0.5f (la mitad de la altura acostado) para coincidir con la forma de la cápsula aplanada; en todos los demás estados vuelve al radio original capturado en Awake().

El umbral 0.001f detiene el Lerp. Sin él, el Lerp asintótico nunca alcanzaría el objetivo exacto y el collider oscilaría indefinidamente.

> StartSliding() asigna la altura del collider instantáneamente sin Lerp: _playerCapsuleCollider.height = crouchHeight. Si el Lerp tardase varios frames en reducir el collider, el jugador colisionaría con la geometría baja que intenta atravesar.

3.10 - Salto: ReadJumpInput(), ApplyJump(), ExecuteJump()

Buffer de Salto

Al detectar JumpInputPressed() en Update:

```
        if (JumpInputPressed())
            _remainingJumpBufferTime = Configuration_Configuracion.EnableJumpBuffering ?
Configuration_Configuracion.DurationOfTheJumpBuffering : 0f;
```

El temporizador se decrementa en Time.deltaTime cada frame. ApplyJump() en FixedUpdate verifica _remainingJumpBufferTime > 0f como condición. Si el jugador pulsa saltar antes de aterrizar y el buffer no ha expirado, el salto se ejecuta al tocar el suelo.

Coyote Time

Cuando se detecta el flanco descendente (abandonar el suelo sin saltar) en GroundDetection():

```
        if (!_playerIsOnGround && _playerWasOnGroundLastFrame && !_playerIsPerformingJump)
            _remainingCoyoteTime = Configuration_Configuracion.EnableCoyoteTime ?
Configuration_Configuracion.DurationOfTheCoyoteTime : 0f;
```

ApplyJump() acepta el salto si _playerIsOnGround || (_remainingCoyoteTime > 0f && EnableCoyoteTime). Durante la ventana del Coyote Time, el jugador puede saltar aunque técnicamente esté en el aire.

ExecuteJump()

```
        Vector3 currentVelocity = _playerRigidbody.velocity;
        currentVelocity.y = 0f;
        _playerRigidbody.velocity = currentVelocity;
        _playerRigidbody.AddForce(Vector3.up * Configuration_Configuracion.ForceAppliedWhenJumping,
ForceMode.VelocityChange);

        _remainingJumps--;
        _playerIsPerformingJump = true;
        _jumpInputHoldTime = 0f;
        _remainingJumpBufferTime = 0f;
        _remainingCoyoteTime = 0f;

        _lastBodyStateWhileOnGround = _currentBodyState;
        ChangeBodyState(BodyState.InTheAir, false);
```

El reinicio de velocity.y antes de AddForce es necesario para la consistencia. Si el Buffer del Salto activa el salto exactamente al aterrizar, velocity.y podría ser negativo en ese instante. Sin el reinicio, la fuerza se aplicaría sobre esa velocidad negativa, produciendo un salto menor al esperado. Con el reinicio, todos los saltos parten de velocity.y = 0f y producen la misma altura.

ForceMode.VelocityChange aplica velocidad directamente sin dividir por la masa. ForceMode.Impulse produciría resultados diferentes según la masa del Rigidbody.

Salto Variable

Mientras _playerIsPerformingJump es true, el input del salto está mantenido y _jumpInputHoldTime < MaximumHeldTimeOfTheJumpInput, esto se aplica en cada tick de FixedUpdate:

```
        if (_playerIsPerformingJump && Configuration_Configuracion.EnableVariableJump && JumpInputHeld())
        {
            if (_jumpInputHoldTime < Configuration_Configuracion.MaximumHeldTimeOfTheJumpInput)
            {
                _playerRigidbody.AddForce(Vector3.up *
Configuration_Configuracion.ExtraForcePerSecondOfTheVariableJump, ForceMode.Acceleration);
                _jumpInputHoldTime += Time.fixedDeltaTime;
            }
        }
```

ForceMode.Acceleration es una fuerza continua por segundo, independiente de la masa. Cuando se suelta el botón, ReadJumpInput() detecta JumpInputReleased() y fuerza _jumpInputHoldTime = MaximumHeldTimeOfTheJumpInput + 1f, cortando la fase extra en el siguiente tick.

TransitionToStandingAndJump() - corrutina

Saltar desde el estado acostado requiere esperar a que el CapsuleCollider alcance la altura de pie antes de aplicar la fuerza:

```
        ChangeBodyState(BodyState.Standing, false);
        float waitedTime = 0f;

        while (Mathf.Abs(_playerCapsuleCollider.height -
Configuration_Configuracion.CapsuleColliderHeightWhileStanding) > 0.15f && waitedTime < 0.5f)
        {
            waitedTime += Time.deltaTime;
            yield return null;
        }

        ExecuteJump(fromProneState: true);
        _systemWaitingForTransitionToPerformJump = false;
```

El timeout de 0.5f previene un bucle infinito si un techo bloquea la transición. La referencia _proneJumpCoroutineActive permite cancelar la corrutina con StopCoroutine() si la situación cambia antes de que se complete.

3.11 - UpdateStamina() y SpendStamina()

UpdateStamina() se ejecuta en FixedUpdate. La detección del movimiento horizontal usa la magnitud al cuadrado para evitar un Mathf.Sqrt() innecesario:

```
        bool playerIsMoving = (_playerRigidbody.velocity.x * _playerRigidbody.velocity.x +
_playerRigidbody.velocity.z * _playerRigidbody.velocity.z) > 0.01f;
```

Si el jugador está corriendo y el coste está activo, llama a SpendStamina(Cost × Time.fixedDeltaTime) y retorna. En cualquier otro caso, acumula _timeElapsedWithoutSpendingStamina. Cuando supera el delay configurado, regenera:

```
            PlayerCurrentStamina = Mathf.Min(PlayerCurrentStamina + regenerationRate * Time.fixedDeltaTime,
Configuration_Configuracion.MaximumPlayerStamina);
```

SpendStamina() es el único punto de escritura para el consumo en todo el sistema:

```
    private void SpendStamina(float cantidad)
    {
```

```
        PlayerCurrentStamina = Mathf.Max(PlayerCurrentStamina - cantidad, 0f);
        _timeElapsedWithoutSpendingStamina = 0f;
    }
```

El reinicio del temporizador garantiza que el delay de la regeneración siempre reinicia desde cero en cualquier gasto, independientemente del subsistema que lo llame (correr, saltar, deslizamiento, dash, postura).

3.12 - UpdateStaminaHUD()

La barra recibe dos actualizaciones paralelas por frame, fillAmount y ancho del rectTransform, ambas mediante Lerp a velocidad 12f:

```
                float proportion = PlayerCurrentStamina / Configuration_Configuracion.MaximumPlayerStamina;
                CurrentStaminaBarImage_ImagenDeLaBarraDeResistenciaActual.fillAmount =
                    Mathf.Lerp(CurrentStaminaBarImage_ImagenDeLaBarraDeResistenciaActual.fillAmount, proportion,
Time.deltaTime * 12f);

                if (_originalWidthInPixelsOfStaminaBar > 0f)
                {
                    Vector2 size =
CurrentStaminaBarImage_ImagenDeLaBarraDeResistenciaActual.rectTransform.sizeDelta;
                    size.x = Mathf.Lerp(size.x, _originalWidthInPixelsOfStaminaBar * proportion, Time.deltaTime *
12f);
                    CurrentStaminaBarImage_ImagenDeLaBarraDeResistenciaActual.rectTransform.sizeDelta = size;
                }
```

_originalWidthInPixelsOfStaminaBar se guarda en Start() con rectTransform.sizeDelta.x y es la referencia base para que el ancho sea siempre proporcional al valor correcto, independientemente de si el rectTransform fue redimensionado en el Editor.

3.13 - ReadCameraInput()

La rotación horizontal rota el cuerpo completo del jugador. La rotación vertical rota solo el CameraSupport. Ambos valores de sensibilidad se multiplican primero por los modificadores activos (zoom, deslizamiento) antes de aplicarse:

```
        float sensitivityH = Configuration_Configuracion.HorizontalMouseSensitivity;
        float sensitivityV = Configuration_Configuracion.VerticalMouseSensitivity;

        if (_zoomIsActive && Configuration_Configuracion.ReduceSensitivityDuringZoom)
        {
            sensitivityH *= Configuration_Configuracion.SensitivityMultiplierDuringZoom;
            sensitivityV *= Configuration_Configuracion.SensitivityMultiplierDuringZoom;
        }

        if (_playerIsSliding && Configuration_Configuracion.ReduceCameraSensitivityDuringSliding)
        {
            sensitivityH *= Configuration_Configuracion.CameraSensitivityMultiplierDuringSliding;
            sensitivityV *= Configuration_Configuracion.CameraSensitivityMultiplierDuringSliding;
        }

        transform.Rotate(Vector3.up * inputX);

        _cameraAccumulatedVerticalRotationAngle -= inputY;
        _cameraAccumulatedVerticalRotationAngle = Mathf.Clamp(
            _cameraAccumulatedVerticalRotationAngle,
            -Configuration_Configuracion.UpperVerticalLimitOfTheCamera,
            Configuration_Configuracion.LowerVerticalLimitOfTheCamera);

        CameraSupport_SoporteDeLaCamara.localRotation = Quaternion.Euler(_cameraAccumulatedVerticalRotationAngle
+ _breathingPitchOffset, 0f, 0f);
```

El ángulo vertical se acumula en un float en lugar de leer la rotación actual del rig, porque las rotaciones Quaternion no pueden sumarse directamente de forma predecible. El acumulador float permite que Mathf.Clamp se aplique correctamente y que Quaternion.Euler se asigne en cada frame.

Los ejes RSHorizontal y RSVertical se leen dentro de bloques try/catch vacíos. Si PlayerControllerSetup aún no se ha ejecutado (ej. primera importación del proyecto), Input.GetAxisRaw() con un eje no registrado lanza una ArgumentException. El try/catch silencioso deja los valores en 0f sin crashear.

3.14 - UpdateCameraHeight()

La posición Y del CameraSupport es la suma de tres componentes independientes: la altura base (interpolada por el estado corporal), el desplazamiento del balanceo continuo y el impulso del balanceo reactivo. Para interpolar solo la base sin verse afectado por los desplazamientos de balanceo, primero se extrae la Y base limpia y luego se recompone:

```
        Vector3 position = CameraSupport_SoporteDeLaCamara.localPosition;

        float yBase = Mathf.Lerp(
            position.y - _continuousBobbingOffset.y - _reactiveBobbingImpulse.y - _breathingTranslationOffset,
            targetHeight,
            Time.deltaTime * Configuration_Configuracion.SpeedOfTheCameraHeightTransition);

        position.x = _continuousBobbingOffset.x + _reactiveBobbingImpulse.x;
        position.y = yBase + _continuousBobbingOffset.y + _reactiveBobbingImpulse.y +
_breathingTranslationOffset;
        position.z = _continuousBobbingOffset.z + _reactiveBobbingImpulse.z;

        CameraSupport_SoporteDeLaCamara.localPosition = position;
```

3.15 - UpdateHeadBobbing()

La velocidad horizontal se calcula con una magnitud manual para excluir el componente vertical, igual que en la detección del movimiento de la resistencia:

```
        float horizontalSpeed = Mathf.Sqrt(
            _playerRigidbody.velocity.x * _playerRigidbody.velocity.x +
            _playerRigidbody.velocity.z * _playerRigidbody.velocity.z);
```

El balanceo continuo usa dos funciones seno a diferentes frecuencias:

```
        if (playerIsMovingOnGround)
        {
            float baseIntensity;
            float baseFrequency;

            if (playerIsRunningNow && Configuration_Configuracion.EnableHeadBobbingWhileThePlayerRuns)
            {
                baseIntensity = Configuration_Configuracion.IntensityOfHeadBobbingWhileThePlayerRuns;
                baseFrequency = Configuration_Configuracion.FrequencyOfHeadBobbingWhileThePlayerRuns;
            }
            else if (Configuration_Configuracion.EnableHeadBobbingWhileThePlayerWalks)
            {
                baseIntensity = Configuration_Configuracion.IntensityOfHeadBobbingWhileThePlayerWalks;
                baseFrequency = Configuration_Configuracion.FrequencyOfHeadBobbingWhileThePlayerWalks;
            }
            else
            {
                baseIntensity = 0f;
                baseFrequency = Configuration_Configuracion.FrequencyOfHeadBobbingWhileThePlayerWalks;
            }

            // ── Multiplicador por postura / Posture multiplier ──────────────────
            if (_currentBodyState == BodyState.Crouching)
            {
                baseIntensity *= 0.75f;
                baseFrequency *= 0.75f;
            }
            else if (_currentBodyState == BodyState.Prone)
            {
                baseIntensity *= 0.3f;
```

```
                baseFrequency *= 0.6f;
            }

            _continuousBobbingTimer += Time.deltaTime * baseFrequency;
            _continuousBobbingOffset.y = Mathf.Sin(_continuousBobbingTimer) * baseIntensity;
            _continuousBobbingOffset.x = Mathf.Cos(_continuousBobbingTimer * 0.5f) * baseIntensity * 0.5f;
        }
        else
        {
            _continuousBobbingTimer = 0f;
            _continuousBobbingOffset = Vector3.Lerp(
                _continuousBobbingOffset,
                Vector3.zero,
                Time.deltaTime *
Configuration_Configuracion.ReturnSpeedOfReactiveHeadBobbingToTheNeutralPosition);
        }

        _breathingTranslationOffset = Mathf.Lerp(_breathingTranslationOffset, breathingTranslationTarget,
Time.deltaTime * 3f);

        _reactiveBobbingImpulse = Vector3.Lerp(
            _reactiveBobbingImpulse,
            Vector3.zero,
            Time.deltaTime * Configuration_Configuracion.ReturnSpeedOfReactiveHeadBobbingToTheNeutralPosition);
```

Esto produce una oscilación elíptica: por cada dos ciclos verticales, ocurre un ciclo horizontal. El resultado es un movimiento natural de cabeza en forma de ocho en lugar de un simple rebote vertical.

El sistema de respiración usa un valor de influencia del agotamiento impulsado por la resistencia:

```
        float breathingPitchTarget = 0f;
        float breathingTranslationTarget = 0f;

        if (Configuration_Configuracion.EnableBreathingEffect)
        {
            float rawExhaustion = Configuration_Configuracion.EnableTheStaminaSystem
                ? 1f - (PlayerCurrentStamina / Configuration_Configuracion.MaximumPlayerStamina)
                : 0f;

            float fadeSpeed = rawExhaustion > _smoothedExhaustionInfluence
                ? Configuration_Configuracion.BreathingExhaustionFadeInSpeed
                : Configuration_Configuracion.BreathingExhaustionFadeOutSpeed;
            _smoothedExhaustionInfluence = Mathf.MoveTowards(_smoothedExhaustionInfluence, rawExhaustion,
fadeSpeed * Time.deltaTime);

            float bpm = Mathf.Lerp(
                Configuration_Configuracion.BreathsPerMinuteAtRest,
                Configuration_Configuracion.BreathsPerMinuteWhenExhausted,
                _smoothedExhaustionInfluence);

            _breathingTimer += Time.deltaTime * (bpm / 60f);
            _breathingTimer %= 1f;

            float inhaleFraction = Configuration_Configuracion.BreathingInhaleFraction;
            float sineInput = _breathingTimer < inhaleFraction
                ? (_breathingTimer / inhaleFraction) * Mathf.PI
                : Mathf.PI + ((_breathingTimer - inhaleFraction) / (1f - inhaleFraction)) * Mathf.PI;

            float wave = Mathf.Sin(sineInput);

            float pitchIntensity = Configuration_Configuracion.BreathingBasePitchIntensity
                + Configuration_Configuracion.BreathingExhaustedPitchIntensity * _smoothedExhaustionInfluence;

            float translationIntensity = Configuration_Configuracion.BreathingBaseTranslationIntensity
                + Configuration_Configuracion.BreathingExhaustedTranslationIntensity *
_smoothedExhaustionInfluence;

            breathingPitchTarget = -wave * pitchIntensity;
            breathingTranslationTarget = wave * translationIntensity;
        }

        _breathingPitchOffset = Mathf.Lerp(_breathingPitchOffset, breathingPitchTarget, Time.deltaTime * 3f);
```

El desplazamiento del pitch de respiración se aplica a la rotación vertical del CameraSupport en UpdateCameraHeight(), y el desplazamiento de traslación se añade al componente Y junto con los

desplazamientos del balanceo. Esto separa la respiración del ciclo del balanceo de cabeza para que ambos puedan ejecutarse de forma independiente.

Los impulsos reactivos se amortiguan en cada frame:

```
        _reactiveBobbingImpulse = Vector3.Lerp(
            _reactiveBobbingImpulse,
            Vector3.zero,
            Time.deltaTime * Configuration_Configuracion.ReturnSpeedOfReactiveHeadBobbingToTheNeutralPosition);
```

3.16 - ReadZoomInput() y UpdateZoom()

ReadZoomInput() solo escribe _zoomIsActive. UpdateZoom() solo lee ese bool para interpolar el FOV. Esta separación permite activar el zoom desde un script externo sin simular el input.

UpdateZoom() también gestiona el efecto del FOV por velocidad cuando el zoom está inactivo:

```
        if (_zoomIsActive && Configuration_Configuracion.EnableTheZoomSystem)
        {
            targetFov = Configuration_Configuracion.CameraFieldOfViewDuringZoom;
            transitionSpeed = Configuration_Configuracion.SpeedOfTheFieldOfViewTransitionDuringZoom;
        }
        else
        {
            targetFov = Configuration_Configuracion.BaseFieldOfViewOfTheCamera;
            transitionSpeed = Configuration_Configuracion.SpeedOfTheFieldOfViewTransitionDuringZoom;

            if (Configuration_Configuracion.EnableSpeedFOVEffect)
            {
                float horizontalSpeed = new UnityEngine.Vector2(
                    _playerRigidbody.velocity.x,
                    _playerRigidbody.velocity.z).magnitude;

                float speedThreshold = Configuration_Configuracion.BaseSpeedOfThePlayer *
Configuration_Configuracion.SpeedFOVStartPercent;
                float speedMax       = Configuration_Configuracion.BaseSpeedOfThePlayer *
Configuration_Configuracion.SpeedFOVMaxPercent;

                float speedRatio = Mathf.InverseLerp(speedThreshold, speedMax, horizontalSpeed);
                targetFov = Configuration_Configuracion.BaseFieldOfViewOfTheCamera
                            * Mathf.Lerp(1f, Configuration_Configuracion.SpeedFOVMaxMultiplier, speedRatio);

                transitionSpeed = Configuration_Configuracion.SpeedFOVTransitionSpeed;
            }
        }
```

El FOV interpolado se escribe en MainCamera_CamaraPrincipal.fieldOfView al final de Update(), no dentro de UpdateZoom(). Esto centraliza la escritura del FOV en un único punto por frame.

3.17 - Interacción con Objetos

TryToPickUpTheObject()

```
        Ray ray = new Ray(MainCamera_CamaraPrincipal.transform.position,
MainCamera_CamaraPrincipal.transform.forward);
        if (Physics.Raycast(ray, out RaycastHit hit,
Configuration_Configuracion.MaximumDistanceToPickUpAnObject))
        {
            if (hit.collider.CompareTag(Configuration_Configuracion.TagOfPickableObjects))
                PickUpObject(hit.collider.gameObject);
        }
```

El Raycast no tiene LayerMask, por lo que detecta todos los colliders. La verificación del tag viene después. Esto es intencional, usar un LayerMask requeriría que los objetos recogibles estuvieran en una capa específica, mientras que el enfoque de tag les permite coexistir en cualquier capa.

PickUpObject() y DropObjectInHand()

Al recoger el objeto, su Rigidbody pasa a isKinematic = true y velocity = Vector3.zero. Al soltarlo, vuelve a isKinematic = false y hereda la velocidad actual del jugador:

```
objectRigidbody.isKinematic = true;
objectRigidbody.velocity = Vector3.zero;
if (DisableTheObjectColliderWhenPickingItUp) collider.enabled = false;


objectRigidbody.isKinematic = false;
objectRigidbody.velocity = _playerRigidbody.velocity;
if (_heldObjectCollider != null) _heldObjectCollider.enabled = true;
```

Sin la transferencia de velocidad, el objeto caería verticalmente desde su posición sin heredar el momento horizontal del jugador.

ThrowObjectInHand()

```
 float chargeRatio = _throwInputHoldTime / Configuration_Configuracion.MaximumChargeTimeOfTheObjectThrow;
 float finalForce = Mathf.Lerp(
     Configuration_Configuracion.MinimumObjectThrowForce,
     Configuration_Configuracion.MaximumObjectThrowForce,
     chargeRatio
 );

        if (Physics.Raycast(cameraRay, out RaycastHit hit, 1000f, mask))
            throwDirection = (hit.point - objectToThrow.transform.position).normalized;
        else
            throwDirection = cameraRay.direction;

        DropObjectInHand();

        Rigidbody rb = objectToThrow.GetComponent<Rigidbody>();
        if (rb != null)
        {
            rb.velocity = Vector3.zero;
            if (Configuration_Configuracion.TakeIntoAccountTheMassOfTheObjectWhenThrowingIt)
                rb.AddForce(throwDirection * finalForce, ForceMode.Impulse);
            else
                rb.AddForce(throwDirection * finalForce, ForceMode.VelocityChange);
        }
```

La dirección de lanzamiento no es simplemente camera.forward. Se dispara un Raycast desde la cámara para encontrar el punto de mira real hasta 1000 unidades de distancia (ignorando la capa propia del objeto sostenido). Si golpea algo, throwDirection = (hit.point − object.position).normalized, por lo que el objeto viaja hacia el punto exacto al que apunta el crosshair. Si el rayo no golpea nada, se usa cameraRay.direction como fallback. ForceMode.VelocityChange: un objeto pesado y uno ligero lanzados con la misma carga alcanzan la misma velocidad. ForceMode.Impulse: los objetos más pesados recorren distancias menores con la misma carga.

RotateObjectInHand()

```
        _currentlyHeldObject.transform.Rotate(camTransform.right, -verticalRotationInput * rotationSpeed,
Space.World);
        _currentlyHeldObject.transform.Rotate(camTransform.up, horizontalRotationInput * rotationSpeed,
Space.World);
```

Los ejes de rotación son camTransform.right y camTransform.up, los ejes propios de la cámara, no los ejes fijos del mundo. Se pasa Space.World para que la rotación se aplique en World Space usando esos vectores relativos a la cámara. Esto significa que el objeto se inclina arriba/abajo relativo a donde mira la cámara, y rota izquierda/derecha relativo a la dirección horizontal de la cámara. Space.Self compondría rotaciones relativas a los ejes locales del objeto, produciendo un comportamiento poco intuitivo al inclinarse.

3.18 - Deslizamiento

StartSliding()

Omite ChangeBodyState() y escribe directamente a _currentBodyState = BodyState.Crouching para evitar el cooldown y los costes de postura. También fuerza la altura del CapsuleCollider instantáneamente:

```
        _currentBodyState = BodyState.Crouching;
        _timeOfLastBodyStateTransition = Time.time;
        _crouchToggleModeIsActive = false;

        float crouchHeight = Configuration_Configuracion.CapsuleColliderHeightWhileCrouching;
        _playerCapsuleCollider.height = crouchHeight;
        _playerCapsuleCollider.center = new Vector3(0f, crouchHeight * 0.5f, 0f);
```

ProcessArcadeSliding()

```
        _timeSinceSlidingStarted += Time.fixedDeltaTime;

        float progressRatio = _timeSinceSlidingStarted /
Configuration_Configuracion.DurationOfSlidingInArcadeMode;
        float currentSpeed = Mathf.Lerp(
            Configuration_Configuracion.BaseSpeedOfThePlayer *
Configuration_Configuracion.InitialSpeedMultiplierOfArcadeSliding,
            Configuration_Configuracion.MinimumSpeedToKeepSliding,
            progressRatio);

        _playerRigidbody.velocity = new Vector3(
            _slideDirectionAtStart.x * currentSpeed,
            _playerRigidbody.velocity.y,
            _slideDirectionAtStart.z * currentSpeed);
```

La dirección se congela en _slideDirectionAtStart desde el primer frame. La velocidad se asigna directamente (no AddForce) para ignorar la fricción del PhysicsMaterial y otras fuerzas, garantizando la curva de desaceleración determinista del modo arcade.

ProcessPhysicalSliding()

```
        Vector3 horizontalVelocity = new Vector3(_playerRigidbody.velocity.x, 0f, _playerRigidbody.velocity.z);
        _playerRigidbody.AddForce(-horizontalVelocity *
Configuration_Configuracion.FrictionDuringPhysicalSliding, ForceMode.Acceleration);

        RaycastHit hit;
        if (Physics.Raycast(transform.position, Vector3.down, out hit,
Configuration_Configuracion.CapsuleColliderHeightWhileStanding,
Configuration_Configuracion.LayersThatAreConsideredGround))
        {
            Vector3 surfaceNormal = hit.normal;
            Vector3 slopeDirection = Vector3.ProjectOnPlane(Vector3.down, surfaceNormal).normalized;
            _playerRigidbody.AddForce(slopeDirection *
Configuration_Configuracion.SlopeAccelerationMultiplierDuringSliding, ForceMode.Acceleration);
```

Vector3.ProjectOnPlane(Vector3.down, normal) calcula el componente de la gravedad paralelo al plano de la superficie. En suelo plano (normal = Vector3.up), el resultado es Vector3.zero. En una pendiente, apunta cuesta abajo con una magnitud proporcional al ángulo de inclinación.

3.19 - ExecuteDash() - corrutina

```
    private IEnumerator ExecuteDash()
    {
        _playerIsPerformingDash = true;
        _timeSinceLastDash = 0f;

        Vector3 dashDirection = GetMovementDirection();
        if (dashDirection == Vector3.zero) dashDirection = transform.forward;

        _playerRigidbody.velocity = new Vector3(
            dashDirection.x * Configuration_Configuracion.DashForce,
            _playerRigidbody.velocity.y,
            dashDirection.z * Configuration_Configuracion.DashForce);

        if (Configuration_Configuracion.EnableTheHeadBobbingSystem &&
Configuration_Configuracion.EnableReactiveHeadBobbingWhenDashing)
            _reactiveBobbingImpulse = transform.InverseTransformDirection(dashDirection)
                                      * Configuration_Configuracion.IntensityOfReactiveHeadBobbingWhenDashing;

        yield return new WaitForSeconds(Configuration_Configuracion.DurationOfTheDashImpulse);

        _playerIsPerformingDash = false;
    }
```

La velocidad se sobreescribe directamente para cancelar inmediatamente cualquier velocidad horizontal previa. Solo se preserva velocity.y para no interferir con la gravedad.

transform.InverseTransformDirection(dashDirection) convierte la dirección del dash del espacio de mundo al espacio local del jugador. El impulso del balanceo se aplica en local space, por lo que la cámara se inclina en la dirección relativa del dash, hacia adelante si el dash es hacia adelante, lateral si es lateral.

Durante DurationOfTheDashImpulse segundos, _playerIsPerformingDash = true bloquea ApplyMovement(), evitando que el input normal interrumpa el impulso.

3.20 - Capa de Input: DetectGamepadTriggers() y DetectActiveInputDevice()

DetectGamepadTriggers()

Los gatillos son ejes analógicos, no botones. Para simular GetButtonDown y GetButtonUp, el estado actual se compara con el estado del frame anterior:

```
        _LTWasActiveLastFrame = _LTIsActive;
        _RTWasActiveLastFrame = _RTIsActive;

        _currentLeftTriggerValue = 0f;
        _currentRightTriggerValue = 0f;

        try { _currentLeftTriggerValue = Input.GetAxisRaw("LT"); } catch { }
        try { _currentRightTriggerValue = Input.GetAxisRaw("RT"); } catch { }

        const float triggerThreshold = 0.3f;
        _LTIsActive = _currentLeftTriggerValue > triggerThreshold;
        _RTIsActive = _currentRightTriggerValue > triggerThreshold;

        _LTZoomPressedThisFrame = _LTIsActive && !_LTWasActiveLastFrame;
        _RTReleasedThisFrame = !_RTIsActive && _RTWasActiveLastFrame;
```

El umbral 0.3f evita activaciones accidentales por drift analógico. Las señales _LTZoomPressedThisFrame y _RTReleasedThisFrame son true durante exactamente un frame. En el siguiente tick, cuando _LTWasActiveLastFrame se actualiza con el valor actual, las señales se reinician automáticamente.

DetectActiveInputDevice()

La detección usa solo KeyCodes de JoystickButton (JoystickButton0 a JoystickButton19), nunca ejes. Los ejes estándar Horizontal y Vertical también reciben las teclas de flecha del teclado; usarlos para

detectar el mando produciría falsos positivos. Los KeyCodes de JoystickButton son nativos de Unity y no lanzan excepciones si no hay un mando conectado.

Si hay input de cualquier JoystickButton - _isUsingGamepad = true. Si hay input de teclado o ratón - _isUsingGamepad = false. El cambio es inmediato en el primer frame en que se detecta el nuevo dispositivo.

4. API Pública

Funciones y propiedades públicas expuestas para que otros scripts del proyecto puedan leer o modificar el estado del controlador sin acceder a las variables privadas internas.

> Práctica Recomendada: para leer la resistencia, usa GetNormalizedStamina() en lugar de PlayerCurrentStamina directamente. Para modificarla, usa siempre ModifyStamina(), pasa por SpendStamina() internamente y garantiza que _timeElapsedWithoutSpendingStamina se reinicie.

4.1 - Funciones de Consulta

Función	Return	Description
ThePlayerIsOnTheGround()	bool	Devuelve _playerIsOnGround con el resultado del último Physics.CheckSphere en FixedUpdate.
GetNameOfCurrentBodyState()	string	Devuelve _currentBodyState.ToString(). Valores: "Standing", "Crouching", "Prone", "InTheAir".
GetNormalizedStamina()	float	Devuelve PlayerCurrentStamina / MaximumPlayerStamina. Rango: 0.0–1.0.
ThePlayerHasAnObjectInHand()	bool	Devuelve _currentlyHeldObject != null.
GetTheObjectCurrentlyInHand()	GameObject	Referencia directa a _currentlyHeldObject. Null si no hay ninguno.
TheDashIsAvailable()	bool	Devuelve _timeSinceLastDash >= CooldownBetweenDashUses.
GamepadIsBeingUsed()	bool	Devuelve _isUsingGamepad, detectado en el último frame de Update.

4.2 - Funciones de Modificación

Function	Parámetros	Description
ModifyStamina(float amount)	float amount	amount < 0: llama a SpendStamina(-amount) y reinicia el temporizador de regeneración. amount > 0: añade con Mathf.Min, no puede superar el máximo.
ForceBodyStateChange(string state)	string state	Llama a ChangeBodyState() con voluntaryChange = false, omitiendo el cooldown y los costes. Acepta: "standing", "crouching", "prone" (sin distinción de mayúsculas).
ForceDropObjectInHand()	—	Verifica si _currentlyHeldObject != null. Si es así, delega completamente en DropObjectInHand(), que vuelve a habilitar isKinematic = false, transfiere la velocidad del jugador al objeto y vuelve a activar su collider.

4.3 - Variable Pública

Variable	Type	Description
PlayerCurrentStamina	float [HideInInspector]	Resistencia en tiempo real. [HideInInspector] la oculta del Inspector. Accesible para lectura directa desde otros scripts.

4.4 - Ejemplo de Uso

Acceso al controlador y sus utilidades desde un MonoBehaviour externo:

```
private PlayerController_ControladorDelJugador _player;

void Start()
{
    _player = FindObjectOfType<PlayerController_ControladorDelJugador>();
}

void Update()
{
    float stamina = _player.GetNormalizedStamina();

    _player.ModifyStamina(-25f);
    _player.ForceBodyStateChange("crouching");

    if (_player.ThePlayerHasAnObjectInHand())
        _player.ForceDropObjectInHand();
}
```